

LEA LABS — ANDROID PROJECT 1

Build Campus Tasks

A step-by-step guide to your first Android app in Kotlin and Jetpack Compose.

CODE-ALONG WORKSHOP

What are we building?

Before writing any code, we must agree on exactly what the app needs to do. We are building **Campus Tasks**, a simple tool to manage daily student activities.

- Keep the project to a single screen.
- Focus on understanding the flow of data.
- Build a foundation we can expand later.

The golden rule:

Never start coding without an acceptance checklist.

ACCEPTANCE CHECKLIST

Accept input

A user can type a task title and tap an "Add" button.

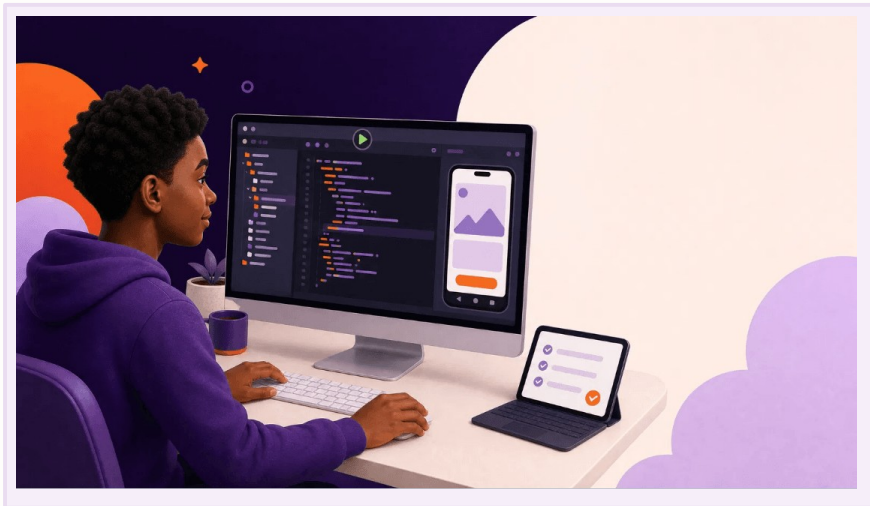
Show a list

The new task appears immediately in a visible list.

Mark as done

A user can tap a checkbox to mark a task complete.

Create the Android Studio project.



- Open Android Studio and select **New Project**.
- Choose **Empty Activity** (Compose) to start with a clean slate.
- Name it **Campus Tasks** and wait for the initial build to finish.

ACTION

Open `MainActivity.kt` . We will write all our beginner code in this single file today.

Create the Task data class.

Every app needs a way to structure its information. In Kotlin, a **data class** is a simple, purpose-built structure to hold values.

- **id**: A unique number to identify the task.
- **title**: The text the user types.
- **isDone**: A true/false flag, starting as false.

```
data class Task( val id: Int, val  
title: String, val isDone:  
Boolean = false )
```

ACTION

Type this code at the very top of
MainActivity.kt , just below the imports.

Remember the screen state.

Jetpack Compose needs to know when data changes so it can redraw the screen automatically.

We use `mutableStateOf` for the single text input, and `mutableStateListOf` for the growing collection of tasks.

MEMORY NOTE

This data lives in memory. It resets when the app closes. We will learn how to save data permanently in a later project.

```
@Composable
fun CampusTasksApp() {
    // Holds the current text in the input
    // field
    var taskName by remember {
        mutableStateOf("")
    }

    // Holds the list of tasks
    val tasks = remember {
        mutableStateListOf<Task>()
    }
}
```

Build the input field.

- Use a **Row** to place the text field and the button side-by-side horizontally.
- The **OutlinedTextField** reads the `taskName` and updates it whenever the user types.
- The **Button** triggers our `addTask()` function when clicked.

ACTION

Add this code inside your main app composable. Run the app on your emulator to test typing into the field.

```
Row(
    VerticalAlignment.Center,
    Alignment.CenterVertically
) {
    (
        OutlinedTextField
        taskName, value =
        it }, onChange = { taskName =
    ) } label = { Text( "New Task"
    )
    (
        Button
    ) } onClick = { addTask
    ) {
    }
    Text( "Add"
    )
}
```

Render the task list.

To display our tasks, we use a **LazyColumn**. It is the standard way to build scrolling lists in Android.

- **Efficient:** It only draws the items currently visible on the screen.
- **Dynamic:** We pass our tasks to the `items()` function.
- **Simple:** For now, we just use a basic `Text` component to show the task name.

ACTION

Run the app. Type a task name, tap "Add", and watch it appear in your scrolling list!

MAINACTIVITY.KT

```
{LazyColumn
    items (tasks) { task ->
        (
            Text
                text = task.name
        )
    }
}
```

Add a tappable task row.

We want a row with a checkbox. Create a **Composable** for each task.

- **TaskRow:** Component for one task.
- **Checkbox:** Shows `isDone`.
- **onCheckedChange:** Notifies when tapped.

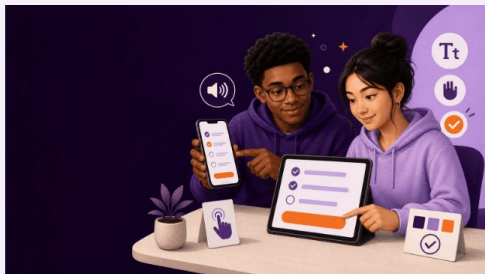
ACTION

Add this function below your main app composable. It defines a single task row.

MAINACTIVITY.KT

```
@Composable
fun TaskRow(task: Task, onCheckedChange: (Boolean) -> Unit) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        Checkbox(checked = task.isDone, onCheckedChange = onCheckedChange)
        Text(text = task.title)
    }
}
```


Combine the pieces.



A Compose app is just functions calling other functions. We place our input field and our list inside a vertical **Column** so they stack neatly on the screen.

PEER REVIEW

Check your code with a partner. Does your overall structure match this outline?

```
@Composable
fun{ CampusTasksApp
    // 1. Screen State
) }var taskName by remember { mutableStateOf ("")
<Task>(tasks = remember { mutableStateListOf

    // 2. Main Vertical Layout
{ Column
    // 3. User Input Area
} Row { /* Input & Button */
    // 4. Scrolling Task List
/* TaskList */ LazyColumn { items (tasks){ task ->
} }
}
}
```

Run, test, and debug.

Testing proves our code works. Debugging is how we learn. Let's verify the app on the emulator and fix the most common beginner mistakes.

TESTING CHECKLIST

- ☐ Type a task name into the input field.
- ☐ Tap the "Add" button and verify the task appears.
- ☐ Tap the checkbox to mark the task as done.
- ☐ Add 15+ tasks to verify the list scrolls smoothly.

BUG: CAN'T TYPE TEXT

SYMPTOM

The keyboard appears, but the text field stays completely empty when you type.

THE FIX

You forgot to update the state! Ensure your text field updates the variable:

```
onValueChange = { taskName =  
it }
```

BUG: LIST WON'T SCROLL

SYMPTOM

When you add many tasks, they disappear off the bottom of the screen and you cannot scroll down.

THE FIX

You used a standard Column. Change it to a LazyColumn to enable scrolling:

```
LazyColumn { items(tasks)  
{ ... } }
```

Improve the project.

You have built a working app from scratch. Now, make it better. Choose a challenge below to push your Kotlin and Compose skills further.

LEVEL 1

Prevent Empty Tasks

Right now, users can add blank tasks. Change your logic to check if `taskName` is empty before adding it to the list.

Hint: Use an `(taskName.isNotBlank())` check before adding.

LEVEL 2

Add a "Clear All" Button

Add a second button next to "Add" that removes all tasks from the list when tapped.

Hint: The `tasks` list has a built-in `.clear()` function.

LEVEL 3

Delete Individual Tasks

Update the `TaskRow` to include a delete icon (X) on the far right. When tapped, remove only that specific task.

Hint: Pass an `onDelete` callback function to `TaskRow`.

Your journey has just begun.

You have successfully built and structured a complete Android application using **Kotlin** and **Jetpack Compose**.

The best way to master these skills is to keep experimenting. Try the challenges, break the code, and learn how to fix it.

See you in the next module.

OFFICIAL RESOURCES

Android Basics with Compose

Google's official training pathway for modern Android development and UI design.

Kotlin Language Guide

Deep dive into the syntax, safety features, and functional concepts of Kotlin.

LEA Labs Mentorship

Share your Campus Tasks app in the community hub for peer and mentor feedback.